

ТЕМА: ДЕРЕВЬЯ



ВВЕДЕНИЕ

Деревья. Казалось бы - все просто: есть корень, есть лист и на этом все. Но это заблуждение, в которое вам не стоит впадать. В этой теме вы узнаете все что требуется для работы с деревьями

УРОК: БИНАРНОЕ ДЕРЕВО

Бинарное дерево

это структура данных, в которой каждый узел имеет не более двух потомков, обычно называемых левым и правым детьми. Дерево имеет корневой узел, с которого начинаются все операции



Основные характеристики

Корень

Узел, не имеющий родителей

Лист

Узел, не имеющий детей

Глубина узла

Расстояние от корня до данного узла

Высота дерева

Глубина самого глубокого узла

Типы бинарных деревьев

Полное бинарное дерево

Каждый узел, кроме листов, имеет два дочерних узла

Сбалансированное бинарное дерево

Глубина левого и правого поддеревьев любого узла отличается не более чем на 1

Дерево поиска

Для каждого узла значение в левом поддереве меньше значения узла, а в правом — больше

Пример реализации бинарного дерева

```
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None

class BinaryTree:
    def __init__(self):
        self.root = None

    def insert(self, value):
        if self.root is None:
            self.root = Node(value)
        else:
            self._insert(self.root, value)

    def _insert(self, current_node, value):
        if value < current_node.value:
            if current_node.left is None:
                current_node.left = Node(value)
            else:
                self._insert(current_node.left, value)
        else:
            if current_node.right is None:
                current_node.right = Node(value)
            else:
                self._insert(current_node.right, value)

    def inorder(self, node):
        if node:
            self.inorder(node.left)
            print(node.value, end=' ')
            self.inorder(node.right)

# Пример использования:
bt = BinaryTree()
bt.insert(5)
bt.insert(3)
bt.insert(7)
bt.inorder(bt.root)  # Вывод: 3 5 7
```

Преимущества и недостатки

Преимущества



Простота реализации и удобство для поиска и сортировки данных

Недостатки



Без балансировки может быть неэффективным при поиске в неотсортированных данных



ВЫВОД

Мы рассмотрели основы бинарных деревьев. Понимание этой концепции поможет вам эффективно управлять и обрабатывать данные в различных приложениях и задачах

